Minha Vida Financeira
v1.0 · Arquitetura

GERAL
Visão Geral
Estrutura de Arquivos
Fluxo de Autenticação

BACKEND
Modelo de Dados
Regras de Segurança
Cache & Performance

FRONTEND
Serviços
Componentes
Sistema de Planos
Módulo de Investimentos

DEV
Setup & Comandos
Decisões Técnicas
Score de Saúde

Arquitetura do Projeto

React 18 + TypeScript | Firebase Firestore | Vite 5 | minha-vida-financeira-a16d2

Visão Geral da Stack

SPA React sem backend próprio — toda a persistência é gerenciada diretamente pelo Firebase SDK no cliente, com Firestore como banco de dados em tempo real e Firebase Auth para autenticação. O servidor Express (`server.ts`) existe exclusivamente para servir a SPA em produção e lidar com webhooks de e-mail via Resend.

React 18
`^18.3.1`
UI declarativa, estado local via hooks

TypeScript
`^5.6.3`
Tipagem estrita em toda a codebase

Vite
`^5.4.10`
Build + HMR em desenvolvimento

Firebase
`^11.0.1`
Auth + Firestore (persistência multi-tab)

Tailwind CSS
`^3.4.14`
Estilização utilitária (classes inline)

Recharts
`^2.13.0`
Gráficos: AreaChart, PieChart

dnd-kit
`^6.3 / ^10.0`
Drag-and-drop de widgets no Dashboard

Resend
`^6.9.3`
Transacional de e-mail via server.ts

Express 5
`^5.2.1`
Servidor para produção + API de e-mail

Grupo Familiar

Recorrentes

Lembretes

Painel Admin

Roadmap

Estrutura de Arquivos

```
minha-vida-financeira/
├── App.tsx ← Estado global, roteamento por aba, auth listener
├── index.tsx ← Entry point React
├── types.ts ← Todos os tipos/interfaces/enums do domínio
├── server.ts ← Express (servir SPA + webhook Resend)
├── firestore.rules ← Regras de segurança do Firestore
├── vite.config.ts
├── tsconfig.json
│
├── services/
│   ├── store.ts ← FirestoreService: CRUD, helpers, formatadores
│   ├── planService.ts ← PlanService: config de planos, feature flags
│   ├── firebase.ts ← Inicialização do SDK + cache persistente
│   ├── healthScore.ts ← Algoritmo score 0-100 em 4 pilares
│   ├── projection.ts ← Projeção de saldo 30/60/90 dias
│   ├── localCache.ts ← LocalCache: pré-hydratação via localStorage
│   ├── landingService.ts ← LandingService: config da landing page
│   ├── brapiService.ts ← Cotações de ativos via API Brapi
│   └── googleAuth.ts ← Helper para login com Google
│
├── components/
│   ├── Layout.tsx ← Shell: sidebar desktop + top bar + bottom nav mobile
│   ├── Dashboard.tsx ← Grade de widgets sortáveis (dnd-kit)
│   ├── TransactionForm.tsx← Modal de criação/edição de transação
│   ├── Profile.tsx ← Perfil, contas, limites, cartão de crédito
│   ├── AdminPanel.tsx ← Painel admin: planos, usuários, landing config
│   ├── LandingPage.tsx ← Página pública (visitante)
│   ├── Login.tsx
│   ├── Register.tsx
│   ├── Onboarding.tsx ← Cadastro pós-registro (nome, contas iniciais)
│   ├── Pricing.tsx
│   ├── HealthScore.tsx ← Anel SVG animado + barras por pilar
│   ├── BalanceProjection.tsx
│   ├── CreditCardPanel.tsx
│   ├── InstallmentTracker.tsx
│   ├── BillReminders.tsx
│   ├── RecurringManager.tsx
│   ├── MonthlyComparison.tsx
│   ├── Investments.tsx
│   ├── GoalVisualizer.tsx
│   ├── FamilyManager.tsx
│   ├── NotificationsView.tsx
│   ├── AdUnit.tsx ← Anúncio exibido para plano Lite
│   │
│   └── components/__tests__/
│       └── *.test.tsx ← Testing Library + Jest
```

Fluxo de Autenticação

A view é controlada pelo estado `authView` em `App.tsx`. `onAuthStateChanged` do Firebase roda uma vez na inicialização e dirige o usuário ao estado correto.

landing → login / register → onboarding → dashboard

```
// App.tsx — onAuthStateChanged bootstrap onAuthStateChanged(auth, async (firebaseUser) => { if (!firebaseUser) { setAuthView('landing'); // não autenticado return; } const profile = await FirestoreService.getProfile(firebaseUser.uid); if (!profile) { setAuthView('onboarding'); // primeiro acesso } else if (profile.isBlocked) { signOut(auth); // conta bloqueada pelo admin } else { setAuthView('dashboard'); // sessão ativa loadUserData(firebaseUser.uid); } });
```

ℹ️ O `LocalCache` pré-popula os arrays de `accounts`, `transactions`, `investments` e `goals` do `localStorage` de forma síncrona antes do primeiro render — o spinner não aparece em visitas repetidas.

🗄️ Modelo de Dados — Firestore

Todas as coleções vivem sob o projeto `minha-vida-financeira-a16d2`. O Firestore usa a hierarquia coleção → documento → subcoleção.

`/users/{uid}`		Documento raiz do usuário
profile	map	firstName, lastName, email, cpf, plan, spendingLimits[], familyMembers[], dashboardOrder[]
subcoleções		

accounts/{id}	subcol	name, type, balance, initialBalance?, creditLimit?, closingDay?, dueDay?
transactions/{id}	subcol	date, description, amount, type, category, accountId, installmentId?, recurringId?, transferId?
investments/{id}	subcol	name, ticker?, assetCategory, investedAmount, currentValue, indexer?, maturityDate?
goals/{id}	subcol	title, targetAmount, currentAmount, deadline, linkedAccountId?
notifications/{id}	subcol	title, message, type, date, read
recurring/{id}	subcol	description, amount, type, frequency, dayOfMonth, lastGeneratedYearMonth, isActive
bill-reminders/{id}	subcol	description, amount, dayOfMonth, daysBeforeAlert, lastNotifiedYearMonth, isActive

/config/{docId}

Configuração global — lida sem autenticação para 'plans' e 'landing'

plans	doc	Configuração dos planos Lite/Premium + features por plano
landing	doc	Textos e configurações da landing page (hero, stats, features, plans)
admins	doc	uids[] — lista de UIDs com acesso admin

/users_index/{uid}

Índice plano para listagem no painel admin

uid	string	Firebase Auth UID
email	string	
name	string	
plan	'lite'|'premium'	
createdAt	string	ISO 8601
isBlocked?	boolean	Bloqueia acesso sem deletar conta
deletedAt?		Soft delete — conta excluída

`deletedAt?`	`string`	Soft-delete — conta na lixeira

> ⚠️ **Saldo âncora**: Contas criadas ou recalibradas a partir de uma certa versão usam `initialBalance` + soma de transações em vez de incremento direto no campo `balance`. Contas legadas (sem `initialBalance`) continuam usando o valor armazenado — veja `computeAccountBalance()` em `store.ts`.

Regras de Segurança

Definidas em `firestore.rules`. Três funções helper: `isOwner(userId)`, `isAdmin()` (verifica `/config/admins.uids[]`), `isFamilyMember(ownerId)` (compara `familyGroupId` no perfil).

Caminho	Read	Write
`/users/{uid}`	owner · familyMember · admin	owner · admin
`/users/{uid}/accounts`	owner · familyMember · admin	owner · admin
`/users/{uid}/transactions`	owner · familyMember · admin	owner · admin; membro cria/edita/deleta as *suas* (createdBy)
`/users/{uid}/investments`	owner · familyMember · admin	owner · admin
`/config/plans · landing`	público (sem auth)	admin
`/config/admins`	autenticado	admin; qualquer auth *cria* se ainda não existir (bootstrap)

`/users_index/{uid}`	owner · admin	admin; owner cria/atualiza o próprio
`/* (demais)`		bloqueado

⚠ **Bootstrap admin**: o email `vcominato@gmail.com` está em `BOOTSTRAP_ADMIN_EMAILS` em `planService.ts`. Na primeira execução, `PlanService.ensureAdminBootstrap()` cria `/config/admins` com o UID correspondente, habilitando o painel admin.

Cache & Performance

Duas camadas independentes de cache garantem que o app carregue instantaneamente em visitas repetidas e continue funcionando offline.

CAMADA 1 — LOCALCACHE (LOCALSTORAGE)

Sincroniza os dados principais do usuário no `localStorage` após cada load do Firestore. No próximo render, os hooks de estado do `App.tsx` são inicializados com esses valores antes do primeiro tick — eliminando o spinner inicial.

```
// App.tsx — inicialização síncrona const [accounts, setAccounts] = useState<Account[]>
(() => { const uid = LocalCache.getLastUid(); return uid ? LocalCache.load(uid).accounts
: []; });
```

CAMADA 2 — FIRESTORE PERSISTENTLOCALCACHE (INDEXEDDB)

O SDK do Firestore é inicializado com `persistentLocalCache + persistentMultipleTabManager`. Isso significa que todas as queries do Firestore são respondidas instantaneamente do IndexedDB, com sincronização em background quando online.

online.

```
// services/firebase.ts export const db = initializeFirestore(app, { localCache: persistentLocalCache({ tabManager: persistentMultipleTabManager(), }), });
```

ℹ️ **Multi-tab:** múltiplas abas do mesmo usuário sincronizam via o `persistentMultipleTabManager`. Uma aba faz a escuta do Firestore; as demais leem do cache local — reduz conexões WebSocket.

⚙️ Camada de Serviços

Toda comunicação com Firebase e APIs externas passa pelos módulos em `services/`. Os componentes nunca importam o SDK do Firebase diretamente.

services/store.ts

CRUD completo para todas as subcoleções do usuário. Helpers de formatação e cálculo de saldo.

`FirestoreService` `formatCurrency` `computeAccountBalance` `localDateStr` `formatDate`

services/planService.ts

Lê /config/plans do Firestore. Expõe hasFeature() para gating. Bootstrap do /config/admins.

`PlanService` `DEFAULT_PLANS_CONFIG` `ALL_FEATURE_MODULES`

services/firebase.ts

Inicialização do SDK (auth + Firestore com IndexedDB). Exporta os singletons usados por todos os outros serviços.

`auth` `db`

services/healthScore.ts

Algoritmo pure-function: 4 pilares (Poupança 30 + Limites 30 + Metas 20 + Diversificação 20).

`computeHealthScore()`

services/projection.ts

Constrói projeção de saldo para 30/60/90 dias com transações futuras + recorrentes + bills.

`buildProjection()`

services/localCache.ts

Wrapper de localStorage para pré-hydratação dos arrays principais antes do primeiro render.

`LocalCache.load()` `LocalCache.save()` `LocalCache.getLastUid()`

services/landingService.ts

Lê e salva /config/landing no Firestore. Usado pelo AdminPanel para editar textos da landing.

`LandingService` `DEFAULT_LANDING_CONFIG`

services/brapiService.ts

Consulta cotações de ações/FIIs via API pública Brapi. Usado em Investments.tsx para preço atual.

`fetchQuote(ticker)`

🧩 Componentes Principais

Estado global vive em `App.tsx` e é passado via props. Não há Context API ou gerenciador de estado externo — mantém o fluxo de dados previsível.

SHELL

Componente	Responsabilidade
`Layout.tsx`	Sidebar fixa (desktop) + top bar + bottom nav (mobile). Recebe `activeTab` + `setActiveTab`.
`Dashboard.tsx`	Grade sortável via dnd-kit. Widgets são componentes independentes renderizados por ID.
`TransactionForm.tsx`	Modal unificado para criar/editar transações, recorrentes e parcelamentos.

WIDGETS DO DASHBOARD (ORDEM PADRÃO)

WIDGETS DO DASHBOARD (ORDEM PADRÃO)

```
// dashboardOrder default em App.tsx [ 'equity', 'monthly-comparison', 'credit-cards',
'health-score', 'installments', 'bills', 'projection', 'ads', 'investments', 'goals',
'accounts', 'notifications', 'cash-flow', 'categories', 'family', 'limits', 'impact' ]
```

PÁGINAS / ABAS

activeTab	Componente	Gate
dashboard	Dashboard.tsx	—
investments	Investments.tsx	feature: investments (Premium)
goals	GoalVisualizer.tsx	feature: goals (Premium)
family	FamilyManager.tsx	feature: family (Premium)
notifications	NotificationsView.tsx	—
profile	Profile.tsx	—
admin	AdminPanel.tsx	isAdmin

💎 Sistema de Planos

Dois planos (`PlanId = 'lite' | 'premium'`). A configuração é armazenada em `/config/plans` no Firestore e carregada na inicialização — o admin pode alterar preços e features sem redeploy.

Lite Premium

Feature	Lite R$ 0/mês	Premium R$ 19/mês
Controle de Transações `transactions`	✓	✓
Gestão de Contas `accounts`	✓	✓
Acesso Mobile e Desktop `mobile`	✓	✓
Sem Anúncios `no_ads`	—	✓
Investimentos `investments`	—	✓
Metas Financeiras `goals`	—	✓
Exportação CSV `csv_export`	—	✓
Gestão Familiar `family`	—	✓

```
// Gating de feature num componente const canUseInvestments = PlanService.hasFeature( plansConfig, user?.plan ?? 'lite', 'investments' ); if (!canUseInvestments) { return <UpgradePrompt feature="investments" />; }
```

📈 Módulo de Investimentos

Implementado em `components/Investments.tsx` (~2.500 linhas) com cotações via `services/brapiService.ts`. Exclusivo do plano Premium (`feature: investments`).

CATEGORIAS E TIPOS

AssetCategory	Tipos suportados	Cotação automática
`VARIABLE_INCOME`	Ações, FIIs, ETF, BDR, Cripto	BRAPI + Yahoo Finance fallback
`FIXED_INCOME`	CDB, Tesouro, LCI/LCA, Poupança	Calculada localmente (ver fórmula abaixo)

MODOS DO FORMULÁRIO

formMode	O que faz
`NEW`	Cadastra novo ativo. Renda variável: qty × preço unitário. Renda fixa: valor aplicado.
`TRADE (BUY)`	Adiciona cotas ao ativo existente. Recalcula custo médio ponderado + cria transação EXPENSE na conta.
`TRADE (SELL)`	Remove cotas proporcionalmente. Reduz custo pelo ratio venda/total + cria transação INCOME na conta.
`ADJUST`	Corrige manualmente quantidade/valor sem gerar transação (recalibração de dados).
`CORPORATE`	Eventos corporativos: renomear ticker, desdobramento, grupamento, conversão M&A.

ESTRATÉGIA DE COTAÇÃO (BRAPISERVICE.TS)

Fonte primária: BRAPI `/quote/list` (endpoint público, sem token, retorna `close` no autocomplete). Fallback: BRAPI `/quote/{ticker}` com token. Fallback final: Yahoo Finance `/v8/finance/chart/{TICKER}.SA` — em dev via proxy Vite, em produção

direto.

```
// Hierarquia de cotação em getStockPrice() 1. BRAPI /quote/list?search=TICKER → campo close // sem token, no autocomplete 2. BRAPI /quote/TICKER?token=VITE_BRAPI_TOKEN // token opcional 3. Yahoo Finance /v8/finance/chart/TICKER.SA // fallback público 4. null → usuário insere o preço manualmente
```

TABELA ESTÁTICA ASSET_DB (BRAPISERVICE.TS)

~120 tickers da B3 (Ações, FIIs, ETFs) com CNPJ da fonte pagadora, nome completo e tipo — usados para preencher automaticamente o formulário sem depender de uma chamada de rede. Prioridade sobre dados retornados pela BRAPI para campos estáticos.

```
// Estrutura da ASSET_DB 'VALE3': { cnpj: '33.592.510/0001-54', name: 'Vale S.A.', type: 'Ações' }, 'KNRI11': { cnpj: '09.072.017/0001-29', name: 'Kinea Renda Imobiliária FII', type: 'FIIs' }, 'BOVA11': { cnpj: '10.406.511/0001-61', name: 'iShares Ibovespa Fundo de Índice', type: 'ETF' },
```

CÁLCULO DE RENDA FIXA (CALCRFESTIMATEDVALUE)

Função pura em `Investments.tsx`. Estima o valor atual de um título sem chamadas de rede — usa apenas os metadados gravados na aplicação.

```
// Fórmulas por indexador Prefixado → invested × (1 + taxa/100) ^ (dias/365) CDI / Selic → invested × (1 + CDI × pct/100) ^ (diasÚteis/252) // diasÚteis ≈ diasCorridos × 252/365 IPCA / IGPM → invested × (1 + (benchmark + spread)/100) ^ (dias/365) // Só aplica se estimativa > custo × 1.00001 (evita regressão por dados incompletos) function getAssetMarketValue(asset) { if (asset.assetCategory === FIXED_INCOME && asset.indexer && asset.indexerRate) { const est = calcRFEstimatedValue(...); if (est > asset.investedAmount * 1.00001) return est; } return asset.currentValue; // fallback: valor salvo manualmente }
```

AUXILIAR DE IR

Modo alternativo da tela que gera os dados para preenchimento da declaração do Imposto de Renda. Calcula valor na posição ano-anterior e ano-atual, proventos no ano selecionado, e gera o texto de discriminação no padrão da Receita Federal.

Tipo de ativo	Código IR sugerido
Ações	`31 - Ações`
FIIs	`73 - FII`
ETF	`74 - ETF`
BDR	`49 - BDRs`
Tesouro / CDB	`45 - Renda Fixa`
LCI/LCA	`45 - Isentos`
Poupança	`41 - Poupança`
Cripto	`01 - Bitcoin`

GRÁFICOS DO MÓDULO

Gráfico	Tipo Recharts	Dado
Alocação Estratégica	`PieChart (donut)`	Valor de mercado por ativo, % do total
Performance Individual	`BarChart`	ROI % por ativo (verde se ≥0, vermelho se <0)
Evolução Patrimonial	`AreaChart`	Patrimônio vs custo investido — 12 meses simulados

Proventos Mensais	BarChart	Total de dividendos/proventos por mês (12 meses)

🚀 Setup & Comandos

PRÉ-REQUISITOS

```
Node.js >= 18 npm >= 9 Conta no Firebase Console (firebase.google.com) Conta no Resend (resend.com) — opcional, só para e-mail transacional
```

INSTALAÇÃO

```
git clone <repo> cd minha-vida-financeira npm install
```

VARIÁVEIS DE AMBIENTE (.ENV.LOCAL)

```
# Opcional — aumenta o limite de requisições na BRAPI VITE_BRAPI_TOKEN=seu_token_brapi # Obrigatório para e-mail transacional (server.ts) RESEND_API_KEY=re_xxxxxxxxxxx # As configurações do Firebase já estão hardcoded em services/firebase.ts # (projeto: minha-vida-financeira-a16d2) — não precisam de .env
```

COMANDOS

Comando	O que faz
`npm run dev`	Inicia o servidor Express + Vite HMR em `localhost:3000`. Firestore é cloud (não local).
`npm run build`	`tsc` + `vite build` — gera `dist/`.

`npm run preview`	Serve o `dist/` localmente para testar o build de produção.
`npm run lint`	`tsc --noEmit` — verifica tipagem sem compilar.

PROXY DE DESENVOLVIMENTO (VITE.CONFIG.TS)

Em dev, as requisições para `/api/yahoo` são proxiadas para `query1.finance.yahoo.com` — evita bloqueio de CORS ao buscar cotações pelo Yahoo Finance.

```
// vite.config.ts server: { proxy: { '/api/yahoo': { target:
'https://query1.finance.yahoo.com', changeOrigin: true, rewrite: (path) =>
path.replace(/^\/api\/yahoo/, ''), } } }
```

BOOTSTRAP DO ADMIN (PRIMEIRA EXECUÇÃO)

ℹ️ Ao fazer login pela primeira vez com `vcominato@gmail.com`, `PlanService.ensureAdminBootstrap()` detecta que `/config/admins` não existe e o cria com o UID do usuário. A partir daí, o painel admin (`activeTab = 'admin'`) fica disponível.

🧠 Decisões Técnicas Não-Óbvias

Escolhas que não são óbvias pelo código e que causariam bugs silenciosos se alteradas sem entender o motivo.

1 · LOCALDATESTR() EM VEZ DE TOISOSTRING()

O Brasil está em UTC-3. `new Date().toISOString()` retorna a data em UTC — após as

21h, UTC avança para o dia seguinte. Uma transação lançada às 22h do dia 15 seria gravada com data "16". `localDateStr()` usa `getFullYear()`, `getMonth()` e `getDate()` que operam no fuso local.

```
// ❌ Errado — pode gravar data errada após 21h no Brasil const date = new Date().toISOString().split('T')[0]; // ✅ Correto — usa fuso horário local do dispositivo const date = localDateStr(); // services/store.ts
```

2 · MODELO DE SALDO ÂNCORA (INITIALBALANCE)

Contas antigas usavam `increment()` no campo `balance` a cada transação. Com o tempo, qualquer inconsistência (transação deletada manualmente no Firestore, bug de concorrência) acumulava drift permanente. O modelo novo grava um `initialBalance` no momento da criação e recalcula o saldo como soma pura a cada render — imune a drift.

```
// computeAccountBalance() em store.ts if (account.initialBalance === undefined) { return account.balance; // legado: usa o valor incremental armazenado } const txSum = transactions .filter(t => t.accountId === account.id) .reduce((sum, t) => sum + (t.type === INCOME ? +t.amount : -t.amount), 0); return account.initialBalance + txSum; // recalculado sempre, sem drift
```

3 · DEDUPLICAÇÃO POR LASTGENERATEDYEARMONTH

Transações recorrentes e lembretes de contas são gerados no carregamento do app. Sem deduplicação, uma segunda abertura do app no mesmo mês geraria duplicatas. O campo `lastGeneratedYearMonth: 'YYYY-MM'` garante que a geração acontece no máximo uma vez por mês.

```
// Padrão de deduplicação (recurring + bill-reminders) const currentMonth = `${year}-${month}`; // ex: '2025-10' if (item.lastGeneratedYearMonth === currentMonth)
```

```
return; // já gerado generateAndSave(item); updateDoc(ref, { lastGeneratedYearMonth: currentMonth });
```

4 · SALDO DE CARTÃO DE CRÉDITO É NEGATIVO

A convenção do app é que `balance` representa o saldo disponível. Para cartão de crédito, débito na conta = dívida aumenta = `balance` fica mais negativo. A UI exibe `Math.abs(Math.min(balance, 0))` como "Fatura atual".

```
// Cartão: balance negativo = dívida // EXPENSE em cartão → balance decreases (dívida cresce) // INCOME em cartão → balance increases (pagamento da fatura) // Exibição no CreditCardPanel: faturaAtual = Math.abs(Math.min(balance, 0)); // 0 se não há dívida limiteDisponivel = creditLimit + Math.min(balance, 0); // limite - dívida
```

5 · ASSET_DB COMO TABELA ESTÁTICA

A BRAPI free tier não retorna CNPJ nem garante nome oficial — dados essenciais para o Auxiliar de IR. A tabela estática `ASSET_DB` em `brapiService.ts` resolve isso offline, sem depender de disponibilidade de API. Atualizações pontuais são feitas diretamente no arquivo.

Score de Saúde Financeira

Calculado em `services/healthScore.ts` a partir dos dados já carregados no estado do app — sem chamada extra ao Firestore. O score varia de 0 a 100 e é composto por quatro pilares independentes.

OS QUATRO PILARES

#	Pilar	Máx.	Dado usado

1	Poupança	30	transactions do mês corrente
2	Limites de Gastos	30	accounts com `limit` definido
3	Metas Financeiras	20	goals (currentAmount / targetAmount)
4	Diversificação	20	accounts + investments

PILAR 1 — POUPANÇA (0–30 PTS)

Calcula a taxa de poupança mensal: `savingsRate = (receitas − despesas) / receitas`. Se não houver receita no mês, retorna 15 pts (neutro).

```
savingsRate ≥ 0.30  →  30 pts
savingsRate ≥ 0.20  →  25 pts
savingsRate ≥ 0.15  →  20 pts
savingsRate ≥ 0.10  →  15 pts
savingsRate ≥ 0.05  →  10 pts
savingsRate ≥ 0.00  →   5 pts
savingsRate  < 0.00  →  max(0, 5 + savingsRate × 20)
```

PILAR 2 — LIMITES DE GASTOS (0–30 PTS)

Para cada conta com limite definido, calcula `gasto / limite` no mês. Média das pontuações individuais. Se não há limites configurados → 15 pts (neutro).

% do limite gasta	Pontos
≤ 80%	30
≤ 100%	20
≤ 120%	10
≤ 150%	5
> 150%	0

PILAR 3 — METAS (0–20 PTS)

`avgPct = média(currentAmount / targetAmount)` de todas as metas. `score = round(avgPct × 20)`. Se não há metas → 10 pts (neutro).

Uma meta é marcada como "em risco" quando: prazo ≤ 6 meses E conclusão < 50%. Esse flag gera uma dica específica no painel.

PILAR 4 — DIVERSIFICAÇÃO (0–20 PTS)

```
≥ 1 conta cadastrada            → +4 pts
≥ 2 contas cadastradas          → +3 pts adicionais
tem investimentos               → +8 pts
RF + RV (ambos os tipos)        → +5 pts
apenas um tipo de investimento  → +2 pts
```

LABELS E CORES

Score	Label	Cor Tailwind
≥ 85	Excelente	emerald
≥ 70	Bom	green
≥ 55	Regular	yellow
≥ 40	Atenção	orange
< 40	Crítico	rose

GERAÇÃO DE DICAS

Cada pilar abaixo de seu máximo gera uma dica contextual. Exemplos de regras:

- Poupança < 20 pts → "Tente poupar pelo menos 10% da sua renda"

- Limite ultrapassado → nomeia a conta específica na dica
- Meta em risco → menciona o nome da meta e o prazo
- Sem investimentos → sugere começar a investir
- Sem RF ou RV → sugere diversificar entre Renda Fixa e Variável

Modelo de Grupo Familiar

Permite que membros da família vejam os dados financeiros de um "dono" e lancem suas próprias transações no contexto compartilhado — sem criar um app separado.

ESTRUTURA DE DADOS

O grupo não é uma coleção própria no Firestore. O vínculo é armazenado no perfil de cada membro:

```
// UserProfile (types.ts)
familyGroupId?: string;   // UID do dono do grupo

// FamilyMember — array em profile.familyMembers[] do dono
{
  id:     string;         // UID do membro (pós-aceite) ou ID gerado
  name:   string;
  email?: string;         // usado para enviar o convite
  role:   'Admin' | 'Membro';
  color:  string;         // cor de avatar no UI
}
```

COMO O ACESSO FUNCIONA

As Firestore Security Rules verificam `myProfile().familyGroupId == ownerId` para

As Firestore Security Rules verificam `myProfile().familyGroupId == ownerId` para conceder acesso de leitura. O membro lê toda a subcoleção do dono, mas só escreve transações próprias:

Subcoleção	Membro pode ler?	Membro pode escrever?
accounts	sim	não
transactions	sim	somente `createdBy == auth.uid`
investments	sim	não
goals	sim	não
recurring	sim	não
bill-reminders	sim	não
notifications	não	não

REGRA FIRESTORE — TRANSAÇÕES

```
match /transactions/{docId} {
  allow read, write:    if isOwner(userId) || isAdmin();
  allow read:           if isFamilyMember(userId);
  allow create:         if isFamilyMember(userId)
                        && request.resource.data.createdBy == request.auth.uid;
  allow update, delete: if isFamilyMember(userId)
                        && resource.data.createdBy == request.auth.uid;
}
```

FLUXO DE CONVITE

1. Dono abre `FamilyManager.tsx` e adiciona membro pelo e-mail.
2. `handleAdd` salva o membro em `profile.familyMembers[]` e chama `handleSendInvite`.
3. App POST para `/api/invite` no Express server (`server.ts`) com o e-mail do convidado.

4. Server envia e-mail via **Resend** com link de cadastro/login.
5. Membro se cadastra e define `familyGroupId = dono.uid` no próprio perfil via onboarding ou link direto.
6. A partir desse momento, as regras Firestore passam a permitir o acesso.

CARREGAMENTO NO APP

Em `App.tsx`, depois de resolver o `onAuthStateChanged`, se o perfil do usuário tiver `familyGroupId`, o app dispara listeners Firestore apontando para `/users/{familyGroupId}/…` em vez de `/users/{uid}/…` — o usuário vê e opera sobre os dados do dono de forma transparente.

Transações Recorrentes

Gerenciadas em `components/RecurringManager.tsx` + `services/store.ts → generateRecurring()`. Cada template gera automaticamente uma transação por mês na subcoleção `/users/{uid}/transactions`.

ESTRUTURA DO TEMPLATE (`RECURRINGTRANSACTION`)

```
interface RecurringTransaction {
  id:                    string;
  description:           string;
  amount:                number;
  type:                  TransactionType;       // 'income' | 'expense'
  category:              string;
  subcategory?:          string;
  accountId:             string;
  frequency:             'monthly' | 'weekly' | 'yearly';
  dayOfMonth:            number;               // 1–28 (seguro para todos os meses)
  startDate:             string;               // YYYY-MM-DD
```

```
  startDate:                string;              // YYYY-MM-DD
  endDate?:                 string;              // YYYY-MM-DD — omitir = eterno
  isActive:                 boolean;
  lastGeneratedYearMonth:   string;              // 'YYYY-MM' — marcador de deduplicação
  familyMemberId?:          string;
  createdAt:                string;
}
```

COMO A GERAÇÃO FUNCIONA

`generateRecurring()` é chamado no bootstrap do app (logo após carregar os dados do Firestore). Para cada template ativo, itera mês a mês de `startDate` até o mês corrente:

1. Monta o ID determinístico `rec-{r.id}-{YYYY-MM}`.
2. Verifica no Firestore se o documento já existe — se sim, pula (deduplicação por ID).
3. Se não existe, cria a transação com `date = YYYY-MM-{dayOfMonth}` (ajustado ao último dia do mês quando necessário).
4. Ao final do loop, persiste `lastGeneratedYearMonth = mês atual` no template — próximos logins não re-escaneiam meses já processados.

DEDUPLICAÇÃO DUPLA

Mecanismo	Como funciona
ID determinístico	`rec-{templateId}-{YYYY-MM}` — mesma chave em toda chamada para o mesmo mês
`lastGeneratedYearMonth`	Marca o último mês processado — evita re-escanear meses passados em logins subsequentes

CAMPOS NA TRANSAÇÃO GERADA

A transação criada herda `description`, `amount`, `type`, `category`, `accountId`, `familyMemberId` do template e recebe `isRecurring: true` + `recurringId: r.id`

para rastreabilidade.

SUPORTE A GRUPO FAMILIAR

Quando o usuário é membro de um grupo, `generateRecurring(txUid, recurrings, recUid)` recebe `txUid = dono.uid` (onde as transações ficam) e `recUid = membro.uid` (onde o `lastGeneratedYearMonth` é atualizado). Isso garante que as recorrentes do membro sejam gravadas no contexto do dono mas o marcador de deduplicação fique no perfil correto.

Lembretes de Contas a Pagar

Gerenciados em `components/BillReminders.tsx`. Funcionam como um calendário de vencimentos recorrentes — ao contrário das Transações Recorrentes, **não criam transações automáticas**; apenas disparam notificações no app.

ESTRUTURA (`BILLREMINDER`)

```
interface BillReminder {
  id:                    string;
  description:           string;
  amount:                number;          // valor estimado
  category:              string;
  dayOfMonth:            number;          // 1–28 — dia de vencimento
  daysBeforeAlert:       number;          // alertar X dias antes (padrão 3)
  isActive:              boolean;
  lastNotifiedYearMonth: string;          // 'YYYY-MM' — evita alerta duplicado
  createdAt:             string;
}
```

LÓGICA DE ALERTA

Executada em `App.tsx → generateBillAlerts()` no bootstrap, após carregar os dados:

1. Ignora lembretes inativos ou cujo `lastNotifiedYearMonth == mês atual`.
2. Calcula `diffDays = dayOfMonth − hoje` no mês corrente.
3. Se `0 ≤ diffDays ≤ daysBeforeAlert`, cria uma `AppNotification` com ID `bill-reminder-{bill.id}-{YYYY-MM}`.
4. Persiste `lastNotifiedYearMonth = mês atual` no lembrete para não re-alertar no mesmo mês.

MENSAGEM DA NOTIFICAÇÃO

```
title:   "Conta vence hoje!"         // ou "amanhã" ou "em N dias"
message: "{description} vence dia {dayOfMonth} — R$ {amount}"
type:    "warning"
```

DIFERENÇA EM RELAÇÃO ÀS RECORRENTES

	Recorrente	Lembrete
Cria transação?	sim, automaticamente	não
Cria notificação?	não	sim
Marcador	`lastGeneratedYearMonth`	`lastNotifiedYearMonth`
Caso de uso	salário, aluguel, assinatura	fatura do cartão, IPTU, IPVA

Painel Admin

Acessado via `components/AdminPanel.tsx` quando `user.profile.plan === 'admin'` ou o e-mail está em `BOOTSTRAP_ADMIN_EMAILS`. Dividido em três abas: **Planos**, **Landing Page** e **Usuários**.

COLEÇÃO USERS_INDEX

Índice leve dos usuários — um documento por UID em `/users_index/{uid}`. Criado no cadastro via `PlanService.registerUserIndex()`. Evita varrer todos os perfis completos para listar usuários no painel.

```
interface UserIndex {
  uid:        string;
  email:      string;
  name:       string;
  plan:       PlanId;          // 'lite' | 'premium'
  createdAt:  string;          // ISO 8601
  isBlocked?: boolean;
  deletedAt?: string;          // definido = está na lixeira (soft-delete)
}
```

ABA PLANOS

Edita `DEFAULT_PLANS_CONFIG` em tempo real — nome, preço e features de cada plano. Salva em `/config/plans`. Afeta todos os usuários imediatamente (sem redeploy).

ABA LANDING PAGE

Edita textos, CTA, depoimentos e imagens exibidos em `LandingPage.tsx`. Salva em `/config/landing`. Esse documento é lido sem autenticação (regra pública no Firestore).

ABA USUÁRIOS

Lista todos os registros de `users_index` (excluindo lixeira). Ações disponíveis:

Ação	O que faz
Alterar plano	`PlanService.setUserPlan(uid, plan)` — atualiza `/users/{uid}/profile.plan` e `/users_index/{uid}.plan`
Bloquear / desbloquear	`setUserBlocked(uid, true/false)` — seta `isBlocked` no perfil e no índice; bloqueio verificado no bootstrap do app
Mover para lixeira	`softDeleteUser(uid)` — define `deletedAt` no índice; dados preservados
Restaurar da lixeira	`restoreUser(uid)` — remove `deletedAt`
Exclusão permanente	`deleteUserFromIndex(uid)` — deleta índice + perfil + subcoleções (accounts, transactions, investments, goals, notifications) via `writeBatch`; **não remove do Firebase Auth**

AUTO-LIMPEZA DA LIXEIRA

`purgeExpiredTrash(30)` é chamado automaticamente ao abrir o painel — exclui permanentemente todos os usuários com `deletedAt` há mais de 30 dias.

BACKFILL DO ÍNDICE

`PlanService.backfillUsersIndex()` varre `/users` e cria entradas em `users_index` para qualquer UID ainda não indexado. Útil para migrar usuários criados